



Assignment Title

HomeFix - Home Maintenance Service System Project Documentation Report

Student Name

Fady Abdelrahman

Student ID

20053593

Lecturer

Dinesh Kanojiya

Module/Subject Title

B8IT1247 – Advanced Web Technologies

Submission Date: 26 April 2026

Academic Year 2025/2026

Acknowledgements & Aspirations

This project represents a humble step toward a future where technology and creativity converge. I believe that every line of code & report written today, and every piece of knowledge gained from **my lecturers and those around me**, is a cornerstone for tomorrow's success.

This work is deeply dedicated to my mother, who bought me my very first computer when I was only 11 years old, at a time when there were many other priorities more important than that. Her sacrifice and vision started this journey. I also dedicate this to everyone who believed in my potential and never stopped supporting me to turn visionary ideas into tangible reality.

The Quote:

"THE FUTURE BELONGS TO THOSE WHO BELIEVE IN THE BEAUTY OF THEIR DREAMS." Eleanor Roosevelt.

Declaration by Students

I, Fady Abdelrahman Mohamed (Student ID: 20053593), hereby declare that the work presented in this report is my original work and has not been previously submitted for academic assessment at this or any other institution. All literature, data, and work done by others and cited within this assignment has been given due acknowledgement and listed in the References section.

Signature: Fady Abdelrahman Mohamed

Date: 26 April 2026

Contents

1. Introduction.....	5
2. System Architecture.....	5
2.1 Layered Architecture.....	5
2.2 Architecture Diagram.....	6
3. Technologies Used.....	7
4. Implementation Details.....	7
4.1 Database Design.....	7
4.2 Key Code Examples.....	8
5. Application Screenshots.....	10
6. Testing.....	13
7. Challenges and Solutions.....	13
8. Conclusion.....	14
9. References.....	15
Figure 1 Home Page.....	10
Figure 2 Home page with featured services.....	10
Figure 3 Services page with Electrical filter active.....	11
Figure 4 Request modal with form fields.....	11
Figure 5 My Requests page showing request list.....	12

1. Introduction

HomeFix is a web-based home maintenance service system that connects customers with professional service providers. The application allows users to browse different types of home maintenance services, submit service requests, track their requests by email, and leave ratings and comments after a service is completed.

The main purpose of this project is to create a practical solution that makes it easier for homeowners to find and book home maintenance services. The system covers six service categories: Electrical, Plumbing, HVAC, Carpentry, Painting, and Appliance Repair. Instead of calling multiple providers, customers can use HomeFix to browse, compare prices, and submit requests through a simple web interface.

This project is a full-stack web application. The backend is built with ASP.NET Core Web API, which provides RESTful endpoints. The frontend uses HTML, CSS, and JavaScript. Data is stored in a SQLite database managed through Entity Framework Core.

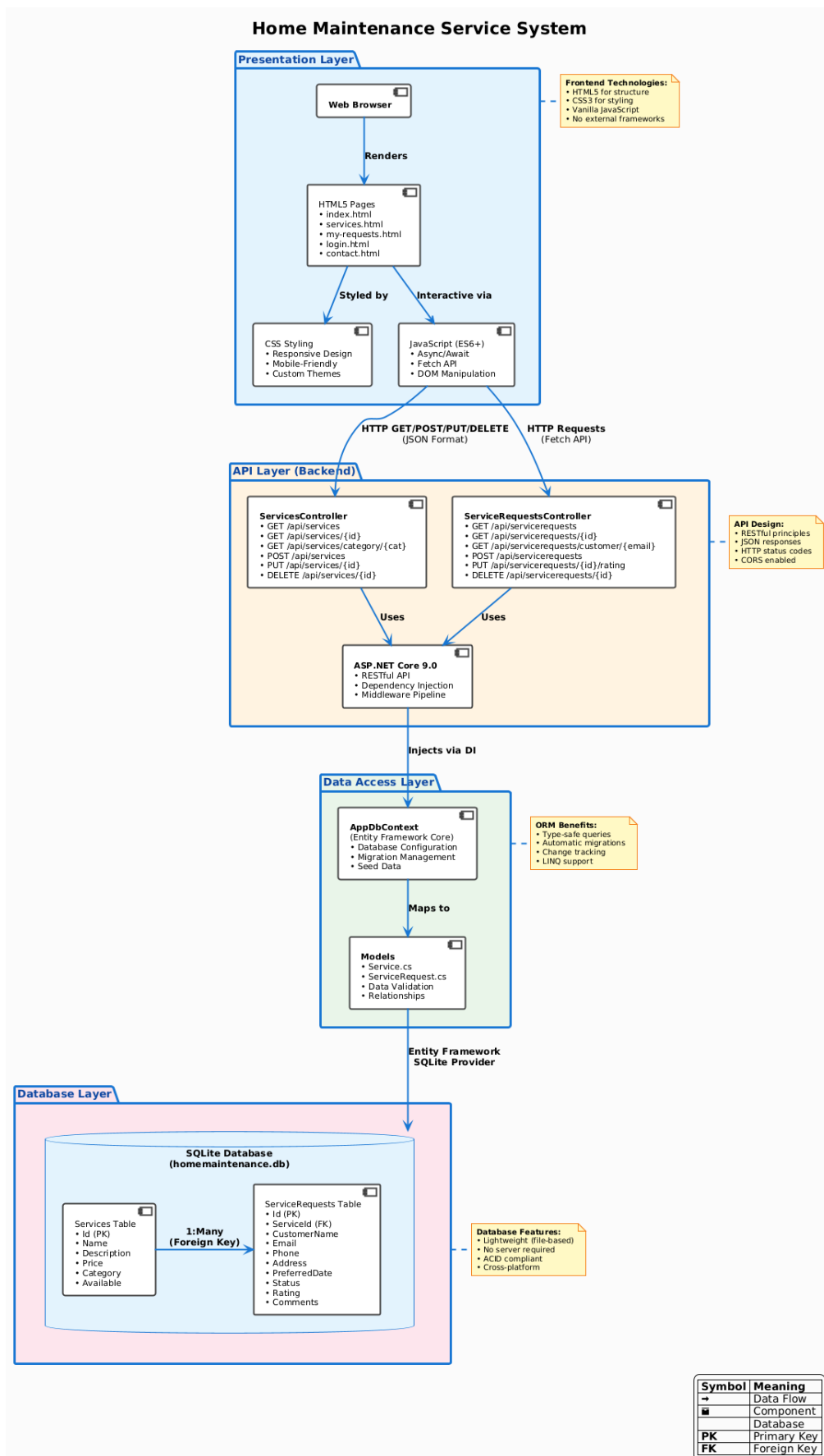
2. System Architecture

2.1 Layered Architecture

HomeFix follows a four-layer architecture that separates responsibilities clearly:

- **Presentation Layer (Frontend):** Five HTML pages `index.html` (home with featured services), `services.html` (all services with filters), `my-requests.html` (track requests by email), `login.html` (authentication), and `contact.html`. JavaScript communicates with the backend using the Fetch API.
- **API Layer (Controllers):** Two controllers handle HTTP requests `ServicesController` for service management and `ServiceRequestsController` for customer requests. Both follow RESTful principles using GET, POST, PUT, and DELETE methods and return JSON data.
- **Data Access Layer:** Entity Framework Core handles all database operations through the `AppDbContext` class. It also seeds six initial services when the database is first created.
- **Database Layer:** SQLite stores all data in a single file (`homemaintenance.db`) with two tables: `Services` and `ServiceRequests`.

2.2 Architecture Diagram



The MVC pattern is applied throughout: **Models** (Service.cs, ServiceRequest.cs) define the data structure; **Views** are the HTML pages that display data dynamically; **Controllers** act as the bridge between the two.

3. Technologies Used

Layer	Technology	Purpose
Backend	ASP.NET Core 9.0	Web API framework
Backend	Entity Framework Core 9.0	ORM for database access
Backend	SQLite	Lightweight file-based database
Backend	C#	Server-side programming language
Frontend	HTML5 / CSS3	Page structure and responsive styling
Frontend	JavaScript (ES6+)	Dynamic content and user interaction
Frontend	Fetch API	HTTP communication with the backend

4. Implementation Details

4.1 Database Design

The database has two tables connected by a one-to-many relationship. One service can have many service requests.

Services Table: Id (PK), Name (required, max 100), Description (max 500), Price (decimal), Category (max 50), Available (bool, default true).

ServiceRequests Table: Id (PK), ServiceId (FK), CustomerName, Email, Phone, Address, PreferredDate, Status (default “Pending”), Rating (nullable, 1–5), Comments (nullable), CreatedAt.

4.2 Key Code Examples

Service Model — Service.cs

```
public class Service
{
    [Key]
    public int Id { get; set; }

    [Required]
    [MaxLength(100)]
    public string Name { get; set; } = string.Empty;

    public decimal Price { get; set; }

    [MaxLength(50)]
    public string Category { get; set; } = string.Empty;

    public bool Available { get; set; } = true;
}
```

The [Key] attribute marks Id as the primary key. [Required] ensures Name is never empty, and [MaxLength] limits field length to prevent database errors. Available defaults to true so all new services are active immediately.

GET All Services — ServicesController.cs

```
[HttpGet]
public async Task<ActionResult<IEnumerable<Service>>> GetServices()
{
    return await _context.Services
        .Where(s => s.Available)
        .ToListAsync();
}
```

This endpoint responds to GET /api/services. It filters only available services using Where(), then retrieves them asynchronously with ToListAsync(). ASP.NET Core automatically converts the result to a JSON array and sends it to the client.

Create Service Request — ServiceRequestsController.cs

```
[HttpPost]
public async Task<ActionResult<ServiceRequest>> CreateServiceRequest(
    ServiceRequest serviceRequest)
{
    serviceRequest.Status = "Pending";
    serviceRequest.CreatedAt = DateTime.Now;
    _context.ServiceRequests.Add(serviceRequest);
    await _context.SaveChangesAsync();
    return CreatedAtAction(nameof(GetServiceRequest),
        new { id = serviceRequest.Id }, serviceRequest);
}
```

This method handles POST /api/servicerequests. It automatically sets Status to “Pending” and records the creation time. After saving the database, it returns a 201 Created response with the location of the new resource.

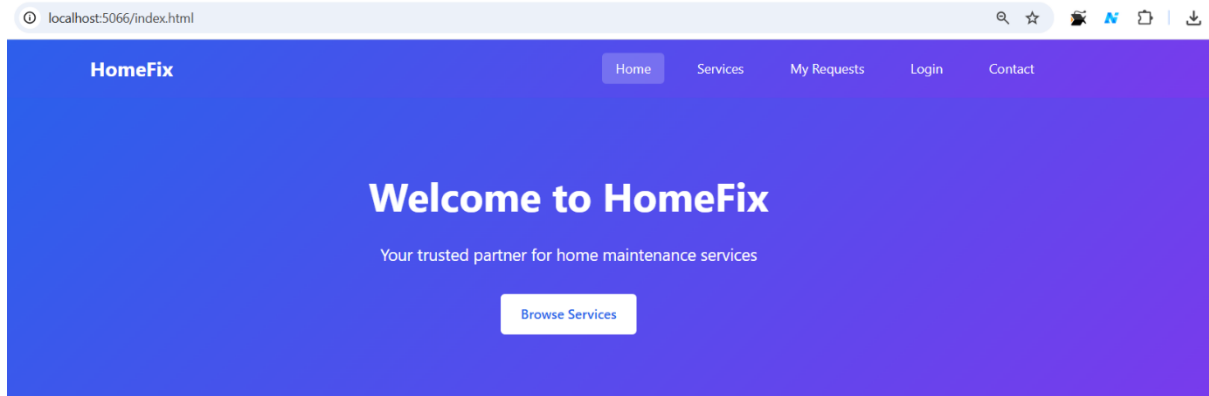
Frontend — fetchAPI Helper (app.js)

```
async function fetchAPI(url, options = {}) {
    const response = await fetch(url, {
        headers: { 'Content-Type': 'application/json', ...options.headers },
        ...options
    });
    if (!response.ok) throw new Error(`HTTP error! status: ${response.status}`);
    return await response.json();
}
```

This reusable function wraps all API calls. It sets the Content-Type header automatically and checks if the response was successful. If not, it throws an error that is caught by the calling function to display a friendly message to the user.

5. Application Screenshots

Screenshot 1 — Home Page (index.html) Shows the HomeFix logo, a welcome banner, and three featured service cards loaded from the API. Each card displays the service name, description, price, and a “Request Service” button.



Why Choose Us

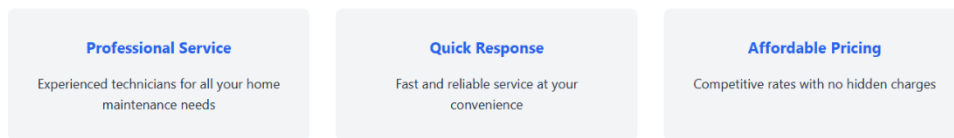


Figure 1 Home Page

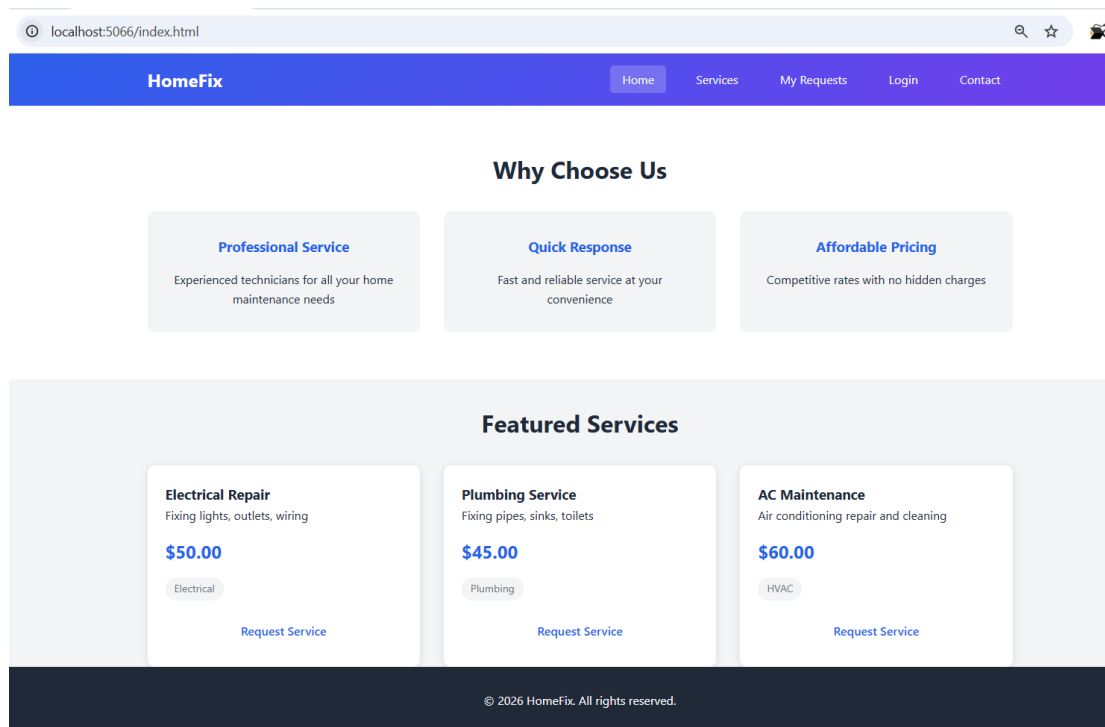


Figure 2 Home page with featured services

Screenshot 2 — Services Page with Filter (services.html) Shows all six services with category filter buttons at the top. When a category is selected, only matching services appear. The active button is visually highlighted.

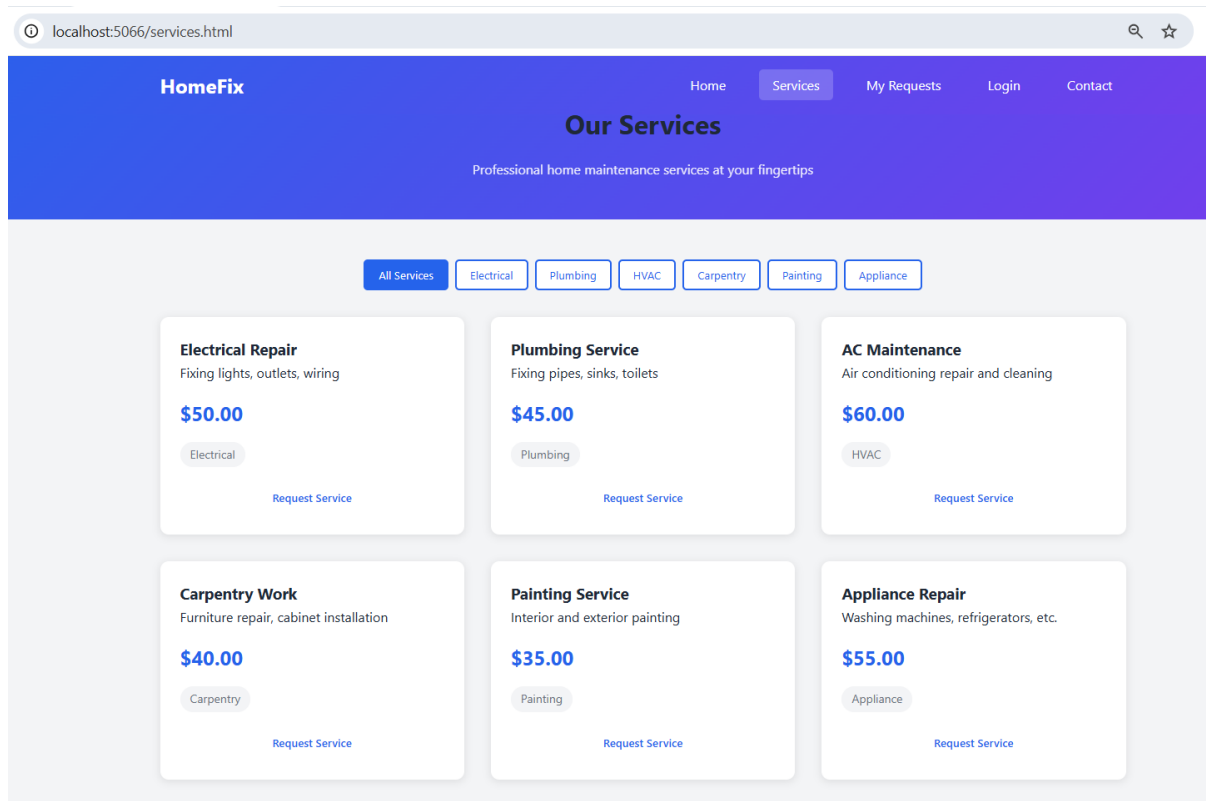


Figure 3 Services page with Electrical filter active

Screenshot 3 — Service Request Modal Shows the popup form that appears when the user clicks “Request Service”. The service name is pre-filled and read-only. The user fills in their name, email, phone, address, and preferred date. Past dates are disabled.

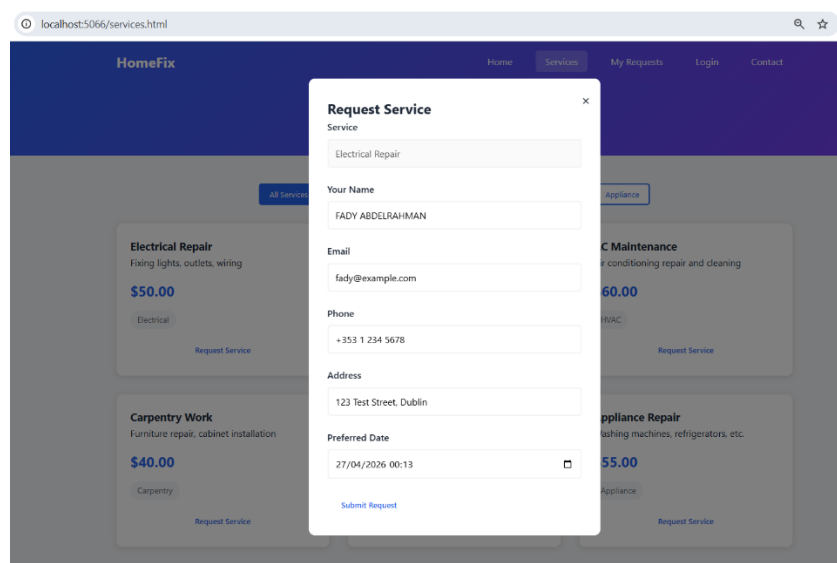


Figure 4 Request modal with form fields

Screenshot 4 — My Requests Page (my-requests.html) Shows the email search field and a list of requests for that customer. Each request shows the service name, date, status badge, and action buttons “Cancel Request” for pending ones and “Rate Service” for completed ones.

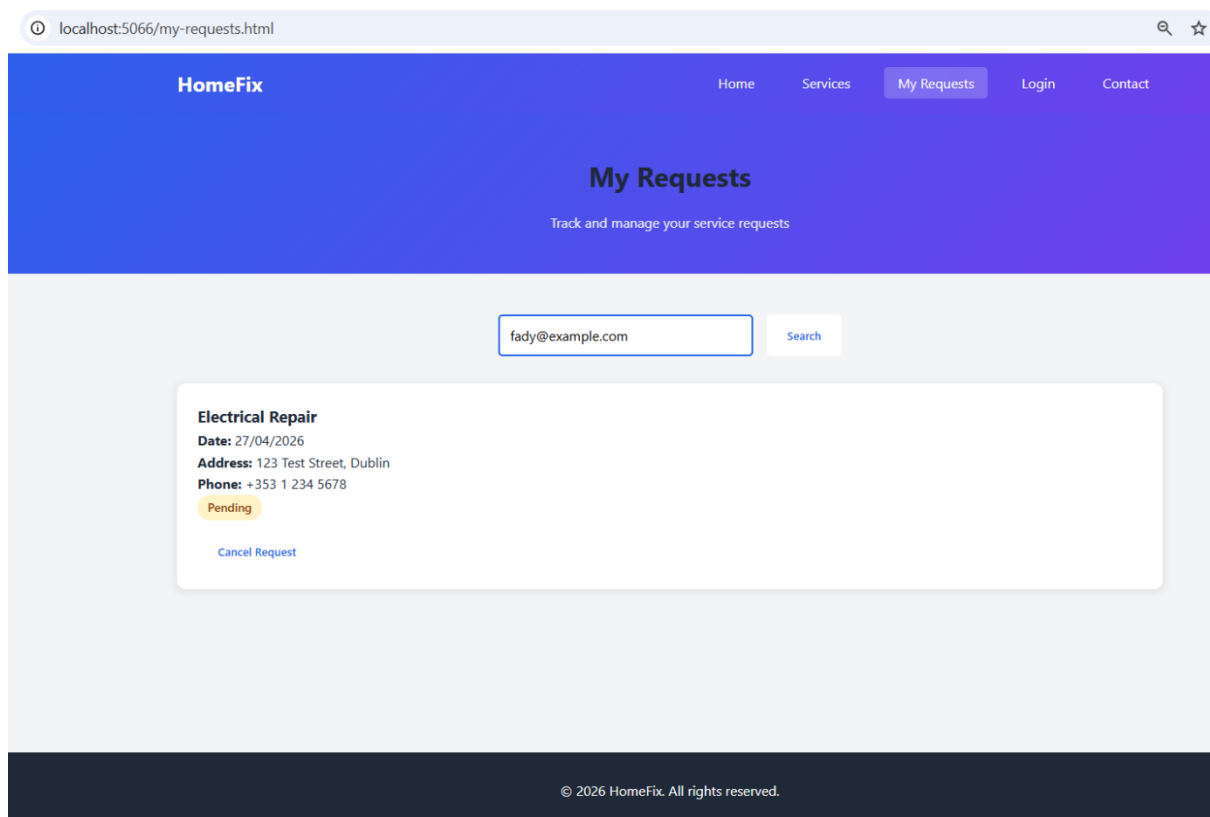


Figure 5 My Requests page showing request list

6. Testing

Test Case	Action	Expected Result	Status
Load Home Page	Navigate to http://localhost:5066	Home page loads with 3 featured services	PASS
View All Services	Click "Services" in navigation	All 6 services displayed with filter buttons	PASS
Filter by Category	Click "Plumbing" filter	Only Plumbing Service shown, button highlighted	PASS
Open Request Modal	Click "Request Service" on a card	Modal opens with pre-filled service name	PASS
Submit Service Request	Fill form with valid data and submit	Success message shown, redirect to My Requests	PASS
Search Requests by Email	Enter email and click Search	All requests for that email displayed	PASS
Cancel Pending Request	Click "Cancel Request" and confirm	Request removed from the list	PASS
Rate Completed Service	Click "Rate Service", select stars, submit	Rating saved and displayed as stars	PASS

All test cases passed. The application handles user input correctly, communicates with the API without errors, and displays data accurately on all pages.

7. Challenges and Solutions

Challenge 1 Async/Await in JavaScript: At the start of the project, the UI was sometimes updated before the data arrived from the API, causing empty sections to appear. After studying the MDN documentation, I understood that all functions using the Fetch API must be marked `async` and `await` must be used to pause execution until the response arrives. Creating the `fetchAPI()` helper function helped standardise all API calls across the application.

Challenge 2 Missing Service Name in Requests: When loading the My Requests page, only the ServiceId number was returned not the service name. I discovered that Entity Framework Core does not automatically load related data. The fix was to use the `.Include(r => r.Service)` method in the controller query, which tells EF Core to join the Services table and include the full service object in the response.

Challenge 3 Preventing Past Date Selection: Users could select past dates in the service request form. I solved this with a simple JavaScript fix in the `openRequestModal()` function: getting today's date in ISO format and setting it as the `min` attribute of the date input field. This disables all past dates in the browser's date picker automatically.

8. Conclusion

The HomeFix project successfully demonstrates a full-stack web application built with ASP.NET Core Web API and JavaScript. The system meets all assignment requirements: two Web APIs with full CRUD operations, proper database design with relationships, user authentication, and a responsive frontend.

Through this project, I gained practical experience in building RESTful APIs, working with Entity Framework Core and SQLite, writing asynchronous JavaScript, and connecting a frontend to a backend over HTTP. Future improvements could include JWT authentication, email notifications for status changes, and cloud deployment on Azure.

9. References

- Freeman, A. (2022). *Pro ASP.NET Core 6: Develop cloud-ready web applications using MVC, Blazor, and Razor Pages*. Apress. <https://doi.org/10.1007/978-1-4842-7957-1>
- Lerman, J., & Miller, R. (2022). *Programming Entity Framework: DbContext* (2nd ed.). O'Reilly Media.
- Microsoft. (2024a). *ASP.NET Core documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/>
- Microsoft. (2024b). *Entity Framework Core — getting started*. Microsoft Learn. <https://learn.microsoft.com/en-us/ef/core/get-started/overview/first-app>
- Microsoft. (2024c). *Create a web API with ASP.NET Core*. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-web-api>
- Microsoft. (2024d). *RESTful web API design best practices*. Azure Architecture Center. <https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design>
- Mozilla Developer Network. (2024a). *Async functions — JavaScript*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function
- Mozilla Developer Network. (2024b). *Using the Fetch API*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- SQLite Development Team. (2024). *SQLite documentation*. <https://www.sqlite.org/docs.html>
- Troelsen, A., & Japikse, P. (2021). *Pro C# 9 with .NET 5: Foundational principles and practices in programming*. Apress. <https://doi.org/10.1007/978-1-4842-6939-8>
- PlantUML (2024). *PlantUML Documentation: Class Diagram*. Available at: <https://plantuml.com/class-diagram>
- Abdelrahman, Fady. (2024).** *Home-Maintenance-Service*. [Source code]. GitHub. Available at: <https://github.com/FadyAbdelrahman/Home-Maintenance-Service> (Accessed: 26 April 2026).

